esprit Se former autrement HONORIS UNITED UNIVERSITIES	**Projet Technologies Web 2A** **2025-2026**	**Étude de cas**

Objectifs :

- Manipuler les balises HTML
- Manipuler les formulaires
- Manipuler les contr.les de saisie HTML
- Manipuler le DOM
- Maitriser les notions de l'OO (Programmation Orienté Objet)
- Création d'une base de donn.es MySQL
- Introduction à l'interface PDO (PHP Data Object)

Enoncé de l'étude de cas : Un réseau d'écoles souhaite moderniser la gestion de sa **bibliothèque scolaire "EspritBook"**. Actuellement, les élèves et enseignants doivent se déplacer pour vérifier la disponibilité des livres, et les responsables de la bibliothèque gèrent tout avec des fichiers Excel.
Objectifs :

- Permettre aux élèves de **consulter en ligne le catalogue** et de **réserver des livres**.
- Offrir aux bibliothécaires un **outil de gestion centralisé** pour gérer les livres et suivre les emprunts et les retours.

Travail à faire :

Développer une application web conviviale et intuitive permettant aux utilisateurs de consulter et rechercher des livres en ligne. L'application doit offrir une expérience utilisateur fluide, tout en permettant à l'équipe de la bibliothèque de gérer facilement le catalogue et les réservations.

- **FrontOffice :**
  - Interface utilisateur attrayante et conviviale permettant aux élèves et enseignants de rechercher des livres par titre, auteur, catégorie, etc.
  - Présentation détaillée des livres avec couverture, résumé, disponibilité et informations sur l'auteur.
- **BackOffice :**
  - Gestion des livres : ajout, modification, suppression et catégorisation par genres/livres.
- **Diagramme de classe :** voici le diagramme de classe simplifié pour illustrer les attributs de l'entité Book.

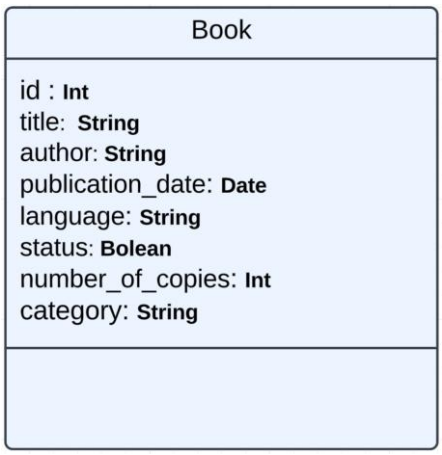


Figure 1 : Table Book

Partie I : HTML

• Arborescence des dossiers et fichiers (html) du projet :

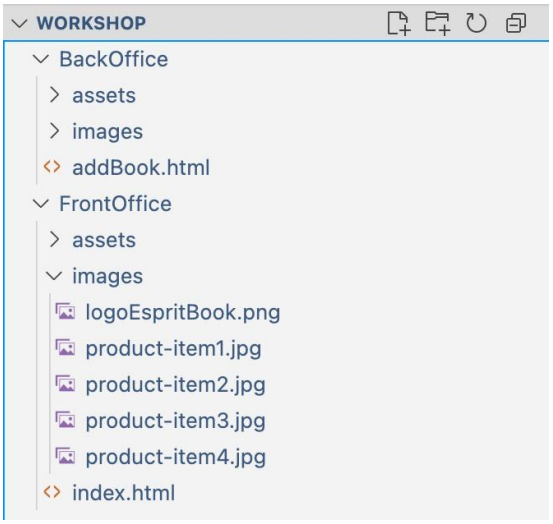


Figure 2: Arborescence des dossiers et fichiers

- ***FRONT OFFICE***

*1. Créer la page **"index.html"** dans le dossier **FrontOffice** comme suit :*

*a. Créer la partie **Header** contenant les liens de navigation entre les pages et Ajouter l'image **LogoEspritBook.png** comme étant un logo.*

- *Library*
- *About*
- *Contact*
- *Dashboard*

Figure 3: Header

*b. Ajouter un entête de page qui contient le nom de l'application "**EspritBook University Library" comme** le montre la figure 4.*

- Utiliser align="center" pour mettre le contenu au centre de la page.

Figure 4 : Header avec nom de l'application

*c. Ajouter une section qui contient la liste des livres **"Digital Collections".***

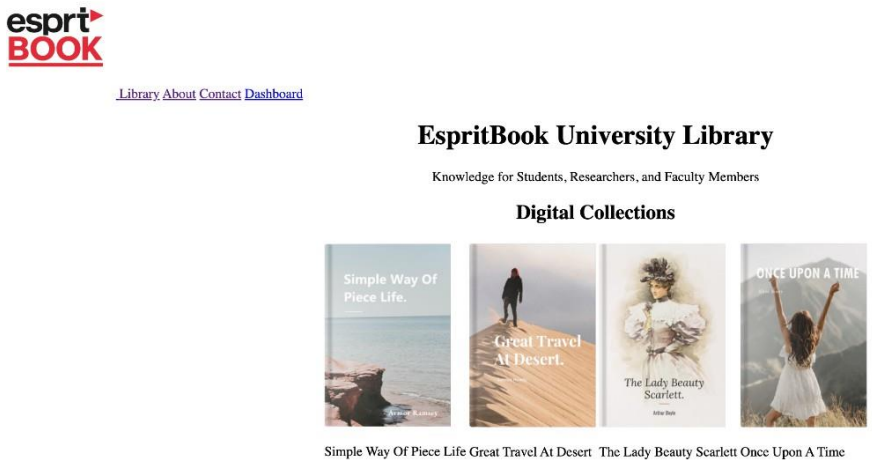


Figure 5 : Section de la liste des livres

d. Ajouter les sections : "**About**" , "**Contact**" et "**Footer**", contenant respectivement une description de l'application, les informations du contact puis un lien e-mail, une adresse et la mention des droits d'auteur **(2025 EspritBook University Library - All Rights Reserved).**

Figure C : Sections about, contact et footer.

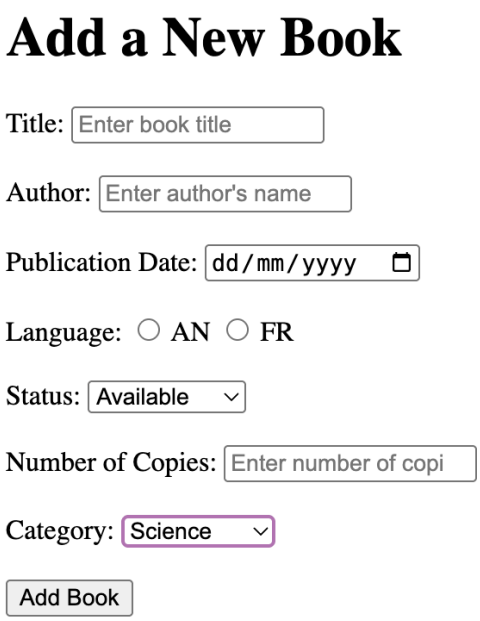
- ***BACK OFFICE***

2. Créer la page **addBook.html** dans le dossier **Backoffice** permettant d'enregistrer un **nouveau livre** dans la bibliothèque. Les conditions suivantes doivent être respectées :

- **Le titre** est obligatoire et doit contenir au moins **3 caractères**.

- **L'auteur** doit contenir uniquement des lettres et espaces, avec une longueur minimale de **3 caractères** (pas de chiffres ni caractères spéciaux).
- **La date de publication** est obligatoire.
- **La langue** doit être limité à deux choix : AN ou *FR*.
- **Le statut** doit être limité à deux choix : *Disponible* ou *Indisponible*.
- **Le nombre d'exemplaires** doit être un entier positif (≥ 1).
- **La catégorie** doit être choisie dans une liste prédéfinie (par exemple : *Science, Literature, Technology, History, Arts*).

Ajouter un **texte indicatif (placeholder)** pour chaque champ afin de guider l'utilisateur.



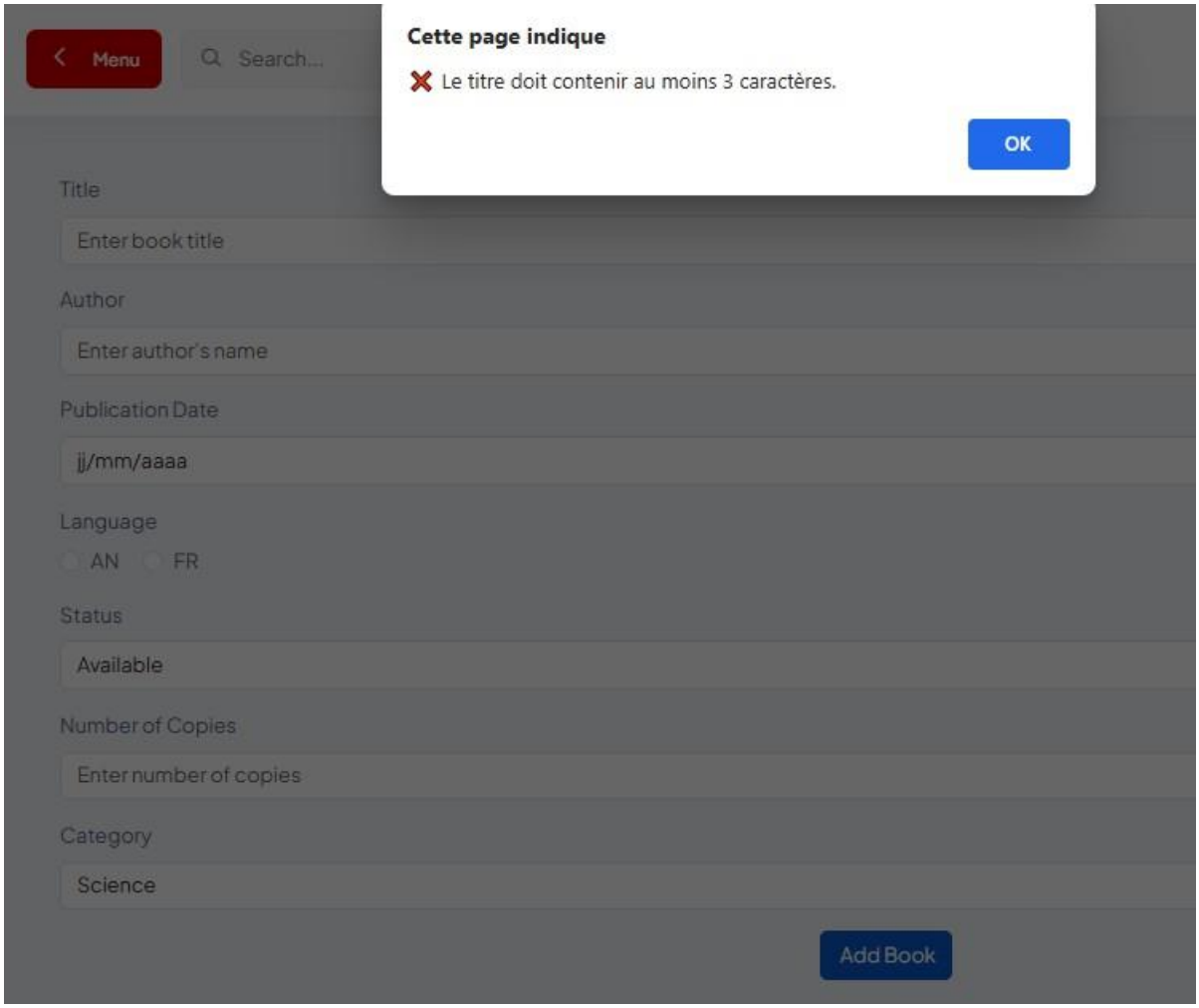
- ***Intégration Template***

3. Utiliser le fichier **"BackFrontOfficeTemplate.zip"**, qui contient deux types de modéles (BackOffice et FrontOffice), pour intégrer un désign à vos fichiers HTML. Vous avez la liberté de concevoir le design selon vos préférences.

Partie II : JavaScript

Partie 1 : Contrôle de saisie avec l'évènement « onClick »

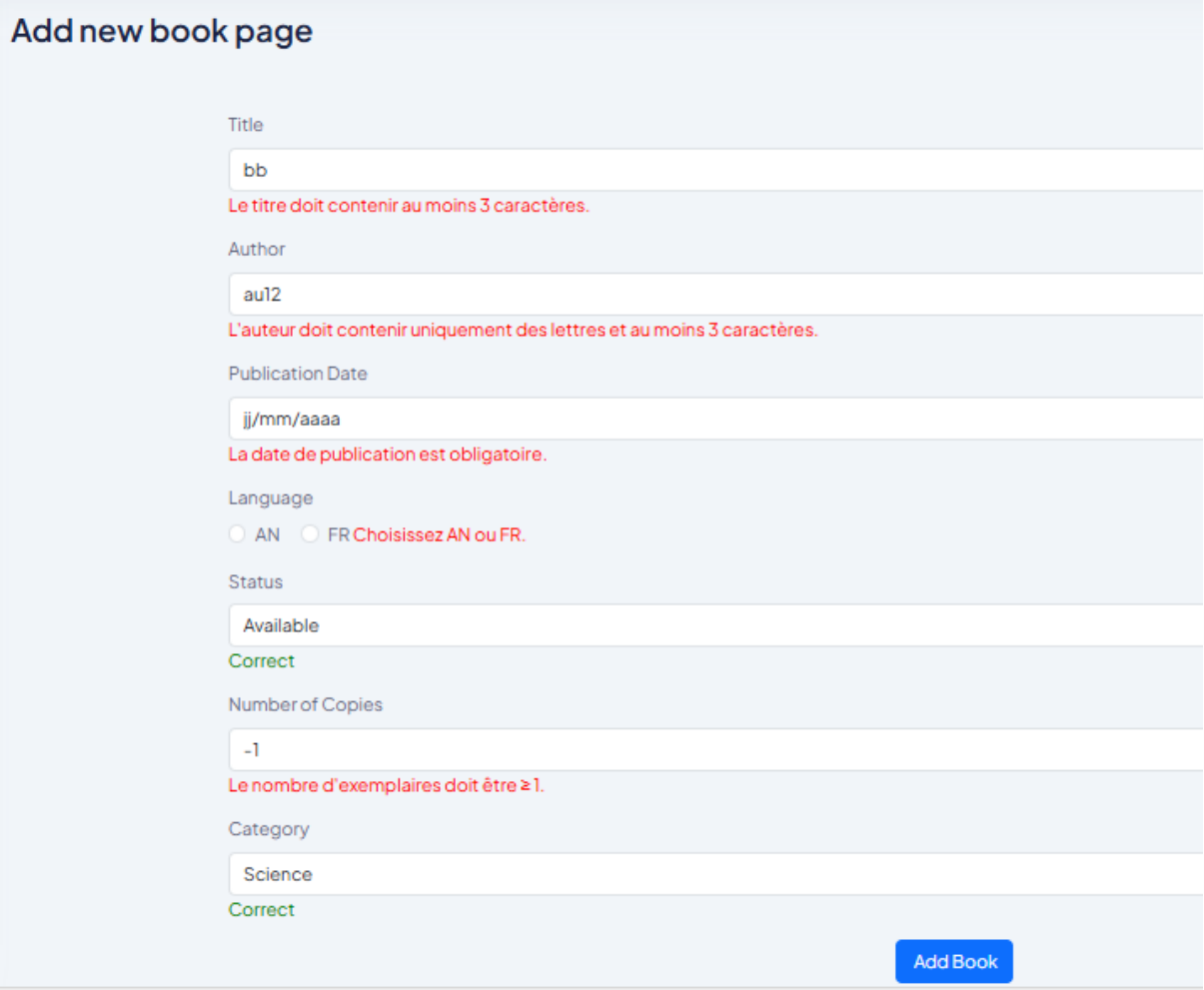
1. Créer un fichier nommé addBook.js.
2. Définir une fonction intitulée « ***validerFormulaire()*** » permettant de contrôler la saisie du formulaire lorsque l'utilisateur clique sur le bouton *"Add Book"*.
   Les critères de validation sont les suivants :
   - Le **titre** doit contenir au moins 3 caractères.
   - L'**auteur** doit contenir uniquement des lettres et des espaces, avec une longueur minimale de 3 caractères.
   - La **date de publication** est obligatoire.
   - La **langue** doit être limitée à deux choix : **AN** ou **FR**.
   - Le **statut** doit être limité à deux choix : **Disponible** ou **Indisponible**.
   - Le **nombre d'exemplaires** doit être un entier positif (≥ 1).
3. Afficher les messages d'erreur dans une boîte de dialogue **alert()**.
4. Appeler la fonction via l'évènement **onClick** du bouton *"Add Book"*.



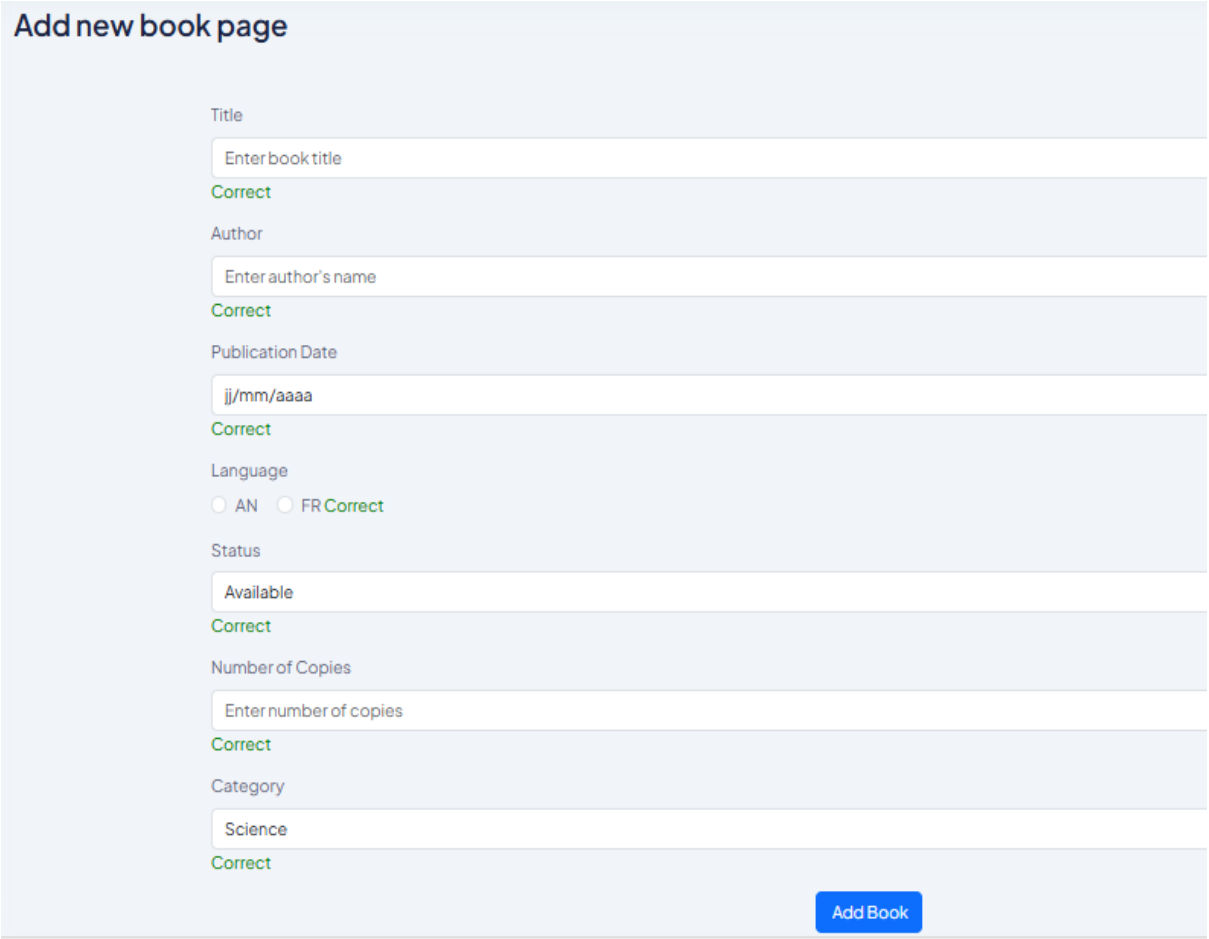
Partie 2 : Contrôle de saisie avec la méthode addEventListener('submit')

1. Ajouter un événement **submit** sur le formulaire avec la méthode **addEventListener('submit')**.
2. Lors de la soumission du formulaire, vérifier les champs selon les critères définis :
   - **Titre** : au moins 3 caractères.
   - **Auteur** : uniquement lettres + espaces (min 3 caractères).
   - **Date de publication** : champ obligatoire.
   - **Langue** : doit être AN ou FR.
   - **Statut** : doit être Disponible ou Indisponible.
   - **Nombre d'exemplaires** : entier positif.
   - **Catégorie** : choisie parmi la liste.

3. Afficher un **message d'erreur en rouge** sous le champ concerné si la saisie est incorrecte.



4. Afficher un **message de succès en vert** sous le champ concerné si la saisie est correcte.



Partie 3 : Contrôle de saisie avec plusieurs événements JavaScript

Implémenter des contrôles de saisie dynamiques à l'aide des événements JavaScript :

- **Champ Title (keyup)** :
  - Vérifier en temps réel que le titre contient **au moins 3 caractères**.
  - Afficher un message d'erreur en rouge si la condition n'est pas respectée, sinon afficher un message de validation en vert.
- **Champ Author (blur)** :
  - Vérifier que l'auteur contient **au moins 3 lettres** et uniquement des lettres ou espaces.
  - Le contrôle se déclenche uniquement lorsque l'utilisateur quitte le champ.
  - Message rouge si invalide, vert si valide.
- **Champ Publication Date (change)** :
  - La date saisie doit être **strictement ultérieure à la date du jour**.

- Message rouge si la date est incorrecte, vert si correcte.

- **Champ Language (change)** :
  - Lorsqu'un utilisateur sélectionne une langue, afficher un message spécifique avec un drapeau :
    - FR en vert pour le français
    - GB en bleu pour l'anglais

Partie III : PHP

Partie 1 : Architecture des fichiers

1. Créer un dossier nommé **EspritBookMVC**.
2. Créer les sous-dossiers suivants :
   - Model : Pour stocker les classes qui représentent les données (comme les livres).
   - **View** : Pour les interfaces qui permettent d'afficher et d'interagir avec les données.
   - **Controller** : Pour la logique métier et la gestion de la communication entre le modèle et la vue.
3. Mettre les dossiers **BackOffice** et **FrontOffice** (utilisés précédemment en HTML/JS) dans le dossier **View.**

```
EspritBookMVC/
├── Model/
|   └── Book.php
├── View/
|   ├── BackOffice/
|   |   └── showBook.php
|   |       addBook.php   (extension modifiée de .html en .php)
|   |       Verification.php
|   └── FrontOffice/
├── Controller/
|   └── BookController.php
```

Partie 2.1 : Manipulation des classes (Couche Modèle)

1. Création de la classe Book
   - Créer un fichier **Book.php** dans le dossier **Model**.
   - Cette classe représente un livre avec plusieurs attributs comme le montre la table **Book**.

Book
Title : String Author : String PublicationDate : Date Langue : String Status : String Copies : int Category : String

2. Créer la méthode show()
- Permet d'afficher les informations relatives à un livre dans un tableau.
- Créer un fichier **showBook.php** dans le dossier **BackOffice**.
- Ce fichier permet de créer un livre (par exemple **book1**) et d'afficher ses informations via :
  - var_dump()
  - la méthode show() définie dans la classe **Book**.
3. Définir un constructeur paramétré et rendre les attributs privés
- Refaire la question 2 avec cette nouvelle version de la classe (encapsulation + getters/setters).

Partie 2.2 : Création d'un objet à partir d'un formulaire

1. Création de la classe BookController
   - Créer un fichier **BookController.php** dans le dossier **Controller**.
   - Cette classe contiendra une méthode showBook($book) qui permet d'afficher les informations d'un livre passé en paramètre.
2. Modifications du formulaire d'ajout d'un livre (addBook.php)
   - La méthode d'envoi des données est **POST**.
   - Les informations sont envoyées au fichier **Verification.php**.
3. **Créer un objet book1** à partir des informations récupérées du formulaire.
4. **Afficher les informations de book1** avec :
   - var_dump()
   - la méthode showBook($book) définie dans le contrôleur.

Partie 3 : CRUD

Partie 3.1 : Configuration

1. A travers l'application **phpMyAdmin**, créer une base de données intitulée « **EspritBook** ».
2. Créer la table Book au niveau de la base de données et mettre l'attribut « **id** » comme une clé primaire de la table.
3. Ajouter des enregistrements à la table via l'application **phpMyAdmin**.
4. Créer la classe « **config** » dans le fichier « **config.php** » sous le dossier parant de votre projet. Au niveau de cette classe, créer la fonction « **getConnexion()** » qui permet d'établir la connexion avec la base de données comme présentée ci-dessous :

```php
<?php
class config
{   private static $pdo = null;
    public static function getConnexion()
    {
        if (!isset(self::$pdo)) {
            $servername="localhost";
            $username="root";
            $password ="password";
            $dbname="myDBName";
            try {
                self::$pdo = new PDO("mysql:host=$servername;dbname=$dbname",
                        $username,
                        $password

                );
                self::$pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
                self::$pdo->setAttribute(PDO::ATTR_DEFAULT_FETCH_MODE,
PDO::FETCH_ASSOC);

                echo "connected successfully";
            } catch (Exception $e) {
                die('Erreur: ' . $e->getMessage());
            }
```

```
        }
        return self::$pdo;
    }
}
config::getConnexion();
?>
```

Partie 3.2: Afficher la liste des livres

1. Dans la classe **BookController,** créer la méthode « **listBooks() »** permettant de récupérer et d'afficher la liste des livres enregistrés dans la table **Book** de la base de données.
2. Afficher la liste des livres au niveau du FrontOffice et BackOffice, faites appel à la méthode «**listBooks() »** afin d'afficher la liste enregistrée dans la table « **Book** ».

Partie 3.3: Supprimer un livre

3. Créer la méthode « **deleteBook($id)** » pour supprimer un livre selon son identifiant passé en paramètre.
4. Ajouter un lien « **delete** » permettant de faire appel à la fonction « **deleteBook($id)** ». Après la suppression, ajouter une redirection vers le fichier de la liste associée.

Partie 3.4: Ajouter un livre

5. Créer la méthode « **addBook($book)** » permettant d'enregistrer un livre passé en paramètre dans la base de données.
6. Créer un fichier « **addBook.php** » au niveau du dossier « **View** » sous le BackOffice.
7. Ajouter une redirection vers le fichier qui contient la liste des livres après l'appel de la fonction d'ajout.

Partie 3.5: Modifier un livre

8. Ajouter un bouton « **update** » qui redirige vers le fichier « **updateBook.php** » en envoyant l'identifiant d'un livre à modifier.
9. Au niveau du fichier « **updateBook.php** », les informations d'un livre à modifier doivent être affichées dans un formulaire. Après modification, les nouvelles informations sont enregistrées dans la base.